BEAM and OTP compatibility: feature matrix and limits

Status: analysis of the current implementation, not a promise of future scope. This note merges the local actor model (D22, D63–D69, D75), the supervision and system-message rules, and the known non-goals already recorded in `docs/roadmap.typ` and `docs/commitments-ledger.typ`.

Short answer

Scale supports a useful subset of BEAM-style actor programming *inside one process*: actors, private mailboxes, channels, links, monitors, a local named registry, selective receive, supervision, and runtime system messages.

Scale does **not** support BEAM distribution. You cannot currently run several Scale instances as one actor system, address an actor on another instance, share a distributed registry, or have all instances execute the same distributed code. Separate Scale processes are independent; there is no remote transport, node identity, remote handle, distributed supervision, or code server.

"All work on the same code" is therefore true only *within one image*: the actors in one run share the append-only image and its code entries. It is not a cross-node code-sharing feature.

The `No` entries in the matrix mean **not implemented today**, not **impossible on .NET**. Most are host or library work. The exceptions are BEAM VM guarantees that .NET does not provide directly: a per-process heap and GC, two-version code purge, and SMP reduction scheduling.

What .NET constrains, and what it does not

A custom scheduler removes the scheduling limit, not every host limit.

- **Scheduling.** The scheduler can add time slices, priority queues, and reduction budgets. It can also drive actors from .NET threads or tasks, which gives parallelism, but not BEAM's millions of preemptively scheduled green processes. Those need a custom stack and runtime, or enough OS threads to change the cost model.
- **Memory.** The .NET garbage collector is process-wide; it does not give each actor a private heap. A per-process heap and GC need a custom allocator or unmanaged memory. The proposed memory tier does not yet provide either.
- **Code reload.** A collectible `AssemblyLoadContext` can load and unload an assembly, but .NET has no two-version module, purge, or release-upgrade model. `Code-Change` replaces a suspended actor's program; it is not a code server.
- **Distribution.** Not blocked by .NET. It needs node identity, authenticated transport, remote capability handles, a serialization and promotion contract, an explicit partition policy, and code/release distribution. None of that is implied by the actor work.
- **OTP surface.** Behaviours, bounded mailboxes, priorities, ETS-like tables, and ports are library or runtime work, not runtime impossibilities.
- **Porting.** A complete .NET port can reproduce the feature set with enough engineering, and it inherits the .NET constraints above. A native language can implement BEAM-style process heaps and preemption directly, but that is a new runtime, not a port of the current interpreter.

The practical conclusion is that feature parity is an engineering programme, not a .NET prohibition. The current scheduler is a starting point; it is not a demonstration that the remaining guarantees follow from it.

What is already in place

- **Actors and scheduling.** `spawn`, `supervise`, `yield`, `halt`, `resume`, and `kill`; per-process scheduler state; cooperative token budgets (D22, D63). Execution is cooperative, not BEAM preemptive scheduling.

- **Communication.** Private mailboxes and store-resident channels; `send`/`receive`; `send-to`/`receive-message`; selective receive `receive-matching` (D65, D68). All of this is local to one run.
- **Failure and observation.** Links deliver abnormal exits; `trap-exit` turns a link exit into a message; monitors fire once; the registry is a local name-to-actor map (D65–D67).
- **Supervision.** `one-for-one`, `one-for-all`, and `rest-for-one`; `permanent`, `transient`, and `temporary` children; restart budgets; token deadlines (D63).
- **System messages.** The reserved runtime class handles Suspend, Resume, Code-Change, Status, and Shutdown before user delivery (D75). This is the `sys`-like control surface, not the OTP code server.
- **Memory.** The image is append-only and shared inside one run; actors do not share mutable state. There is no per-process heap or GC; the proposed memory tier is still an analysis (`docs/memory-and-go-transpiler.typ`).

Feature matrix

BEAM / OTP feature	Scale	What exists	Limit / missing step
Lightweight processes	Partial	Actors with per-actor state, a private mailbox, and a cooperative scheduler	No OS-thread parallelism or SMP; a token budget is not preemption
Message passing	Local	Mailboxes, channels, store-resident queues, `send`/`receive`	No remote send, no wire protocol, no node addressing
Selective receive	Local	`receive-matching` scans oldest-first and blocks to rescan	No receive timeout; one predicate quotation; not inside `quotationRun`
Links	Local	Abnormal exits reach linked actors; trapping is supported	No remote links; link sets are per incarnation and per run
Monitors	Local	One-way `Down` message, fired once; `demonitor`	No remote monitor; no distributed failure detector
Supervision trees	Partial	Three restart strategies, restart types, budgets, token deadlines	No OTP `supervisor` behaviour, child specs, applications, or release tree
Named registry	Local	`register`/`unregister`/`whereis` map a name to a local handle	No global or distributed registry; no name conflict resolution across nodes
System messages	Runtime subset	Suspend, Resume, Code-Change, Status, Shutdown, handled before user code	No `sys` release-upgrade callback or two-version code model
Hot code reload	Partial	Code-Change replaces a suspended actor's program from a quotation	No code server, purge, old/new versions, or release handling
Preemptive scheduling	No	Cooperative yield and token budgets; blocked actors consume nothing	No wall-clock preemption or true parallel scheduling

BEAM / OTP feature	Scale	What exists	Limit / missing step
Per-process heap and GC	No	Shared append-only image; ownership and linear-region analysis	No BEAM process heap or per-process GC; memory tier is not implemented
Distribution / nodes	No	One process-local scheduler and image	No node identity, transport, remote handles, clustering, or partition policy
Remote spawn and send	No	`spawn` and `send` are local	No remote code loading, remote capability, or serialization contract
Distributed registry	No	Registry is local execution state	No global names, replication, or partition-aware lookup
Remote code sharing	No	All actors in one run share the local image	No code server or release distribution across instances
Fault tolerance across nodes	No	Local supervision and restart only	No failover, replication, or split-brain policy
OTP behaviours	No	Primitives and supervision, not `gen_server`/`gen_statem` behaviours	A language-level behaviour layer would be a separate design
ETS / Mnesia	No	Image store and tensor entries	No ETS tables, transactions, or distributed database
Ports / NIFs / external OS processes	No core feature	Host boundary only	No BEAM port protocol or NIF ABI
Bounded mailboxes	No	Mailboxes are unbounded	No backpressure or mailbox limit
Priorities	No	One ready queue; no priority classes	No priority scheduling or per-message priority
Actor migration	No	Actors are local to the run	No migration, placement, or remote relocation
Application / release lifecycle	No	No OTP application or release controller	No start/stop phases, release upgrades, or dependency ordering

Remote instances and shared code

Running several Scale processes is possible, but they do not form a distributed actor system. They cannot currently:

- address each other's actors with a remote handle;
- send a message to a mailbox on another process;
- share a registry or resolve a name across processes;
- supervise a child on another process;
- distribute code or a release to peers;
- decide what happens when a peer disappears or the network partitions.

A future distribution layer would need, in order:

1. node identity and authenticated transport;
2. remote capability handles and a serialization/promotion contract;
3. remote send/receive and mailbox semantics at the boundary;
4. remote spawn/supervision and an explicit partition/failure policy;
5. a distributed registry and name-conflict rules;
6. a code/release distribution and version model;
7. bounded mailboxes, priorities, and backpressure.

None of these are currently implemented, and none are implied by the existing actor or system-message work.

Fit with Scale's architecture

- The append-only image is a *local* authority. Sharing it across nodes is not a message-passing detail; it needs a distribution and code-version model.
- There is no per-process heap, so BEAM's per-process GC and heap isolation do not map directly. The proposed memory tier is a separate design.
- Linear ownership and capabilities are local execution state. Remote handles would need explicit serialization, identity, and revocation rules.
- System messages already separate runtime control from user messages (D75); a future code server would extend that surface, not reuse `send` blindly.

References

- `docs/roadmap.typ` Track C and the D22/D63–D69/D75 sections.
- `docs/commitments-ledger.typ` actor, supervision, and system-message rows.
- `docs/decision-records.typ` D22, D63–D69, D75.
- `docs/memory-and-go-transpiler.typ` for the proposed memory tier.
- `docs/rejected-designs.typ` for the non-goals recorded so far.